Reinforcement learning approach for robotic arm grasping using Isaac Lab.

Problem statement

One of the fundamental requirements of a human service robot is manipulation. Robotics has utility in a care environment, therefore the ability to provide support for those with low mobility and other physical support needs is vital. Deterministic models for manipulation often fail in real-world settings as they cannot adapt to uncertainties. Therefore, reinforcement learning and other machine learning methods are seen as a useful tool for building manipulation behaviours in robots. Reinforcement learning is a machine learning technique where an agent learns from trial-and-error by interacting with an environment and optimising a reward function. For this project I attempted to develop a reinforcement learning algorithm that would result in a robot learning the first step of manipulation, reaching out towards an object. I then wanted to create a python script independent of the reinforcement learning algorithm that could use the policy in a simulation environment.

Background

Markov Decision Processes

As stated above, reinforcement learning is a technique where an agent is trained based on their interactions with an environment. These interactions can be formalised as Markov Decision Processes (MDP). MDPs are a framework for sequential decision making when outcomes are partly random and partly influenced by a decision maker. They are defined by a tuple that contains: the state space, the set of states that represent the environment; the action space, the set of all actions available to the agent from each state; transition probability, a probability function that describes the likelihood that an action in a certain state will lead to another state; the reward function, which calculates the reward received after transitioning between states; and the discount factor, which determines the importance of future rewards relative to immediate rewards. Many reinforcement learning problems can be modelled as MDPs, and most reinforcement learning algorithms are designed to operate using these MDP components.

Reinforcement Learning Algorithms

One of the largest groups of reinforcement learning algorithms are policy gradient methods. These algorithms maintain a neural network known as a policy, which maps a state to probability distributions of the actions it can take. As the agent interacts with the environment, the algorithm will use the rewards to estimate how good each of the actions were. They then update the parameters of the policy network in the direction that increases the expected cumulative reward.

The Proximal Policy Optimisation (PPO) algorithms were a subfamily of policy gradient methods published by researchers at Open AI in 2017 (Schulman et al., 2017). The primary goal of the PPO algorithm is to make the largest step possible to improve performance whilst satisfying a special condition on how close the new and old policy must be. This differentiates them from vanilla policy gradient methods, where the network is improved in each step without a trust region. They are some of the most widely used reinforcement

learning algorithms in modern research (Tan, 2025). PPO algorithms are actor-critic algorithms, meaning that they maintain two networks. These are a policy network to map states to actions, and a value function network which calculates the expected long-term rewards for all the possible actions that can be taken in a state. In each training iteration, the algorithm will first interact with its environment using the policy network, recording the state, action, action probabilities and the reward for each step. It will then estimate the expected return of each state using the value function network. Based on the recorded reward data the algorithm calculates the actual total return for each time step. It then calculates the advantage, which measures how much better or worse an action performed compared to the expected outcome. Using the advantage and the recorded action probabilities, the algorithm calculates the network loss for the policy and value function networks. In this step the PPO objective function clips the loss to keep updates to the network within a small, trusted region. These losses are back propagated through their respective networks, where parameters are shifted to in the direction that reduces loss. Effectively teaching each of the networks to maximise the total return while maintaining stable policy updates.

Relevant studies

In the literature, there are many studies recording the use of Proximal Policy Optimisation for robot manipulation training.

Shahid et al. (2020) trained the Franka Emika Panda Robot in MuJoCo simulation environment to first reach a small cube and then lift it. To split the two subtasks, the researchers used distinct reward functions for each. Where, the end-points proximity to the cube determined which reward to calculate. When the hand had a greater distance than 2.5 cm from the centre of the 6 cm cube, positive rewards were given for the hand's velocity and its distance to the cube. A small positive reward was given if the gripper was open. Then, when the distance was less than 2.5 cm, as it implies that the cube would be within the grasp of the endpoint, the reward function was switched. Positive rewards were given for the distance between the cube and endpoint, the endpoints velocity, and the z-coordinate of the cube. Small rewards were also given if the gripper was closed and if both fingers were in contact with the cube. Researchers were able to train the policy in around 160,000 total episodes.

In robotics, as the environment is constantly changing, there is always large room for error. Reinforcement learning algorithms should be able to punish agents for doing unfavourable actions. Tang et al. (2022) applied PPO algorithms to control dual-arm robots for trajectory planning, enabling them to approach and lift a patient. They used several punishment methods, embedding obstacle avoidance and collision detection into their reward function, giving negative rewards if the joints were near obstacles. Similar punishments would be relevant for our human service manipulation problem, particularly collisions and dropping objects. Tang et al. (2022) also used a small penalty for time. It is important to provide a motive for efficiency, however setting large values for the time punishment has been observed to suppress exploration.

My Solution

To begin the Fourier GR1 manipulation learning process, my solution was first break it up into modular subproblems and then implement reinforcement learning environments to teach the steps. Reaching out to a target point or object is one of the most basic subproblems in manipulation and is used in many greater manipulation tasks. To perform complex reinforcement learning algorithms for manipulation that rely on gravity and other forces, a physics simulator is required. Isaac Sim is a GPU-accelerated engine that enables efficient learning and allows for thousands of parallel environments. Isaac Lab is a reinforcement learning framework that is built on top of Isaac Sim. It provides an interface to run reinforcement learning environments and develop models within the simulator.

Reinforcement Learning Algorithm

As Isaac Lab is a relatively new software platform, released in June 2024, there is not extensive documentation, and not many codebases available in the literature. Therefore, when creating my solution I largely stuck Isaac Lab documentation and tutorials. My solution was created as an Isaac Lab external project, using their project creation tool. I used the PPO algorithm provided by the Robotics Systems Lab's reinforcement learning library (rsl_rl). The PPO algorithm was used as there are many examples of its use in similar manipulation projects in the literature. As my task has added randomness such as with the object's position, the PPO algorithm is very suitable, as it enforces strict limits in change when modifying the network. Using this algorithm, I did not observe any policy collapse when running the environment for an extended number of iterations, the mean reward function would instead converge (Dohare et al., 2023).

In my Isaac Lab environment, the observation space was defined as 28 dimensions. There are 7 joints in each arm, so the angular position and velocity of the left arm were used. Additionally, the world frame position and orientation (pose) of the object and the end effector was used. In my last report the only inputs were the joints and the object world frame position, however, after advice from Dr Francisco Cruz Naranjo, adding the end effector world frame position the efficiency of the algorithm greatly improved. The policy attempts to predict the reward based on the observation state, therefore it is important to include data used in the reward function calculations within the observation space itself where possible.

The action space had a dimension of 7. The policy output is a group of continuous probability functions corresponding to joint targets for the 7 joints in the left arm.

The reward function contained 7 components. The dense reward was the scalar distance of the left end effector and the object's centre. A sparse reward was given when the palm of the robot faced the object. This was calculated by finding a plane between the middle and index finger proximal joints and the wrist joint. Then comparing the normal of the plane to the vector of the end effector (the middle finger proximal joint was used for this). When the dot product was greater than 0.5, then the reward was given. A dot product of 0.5 maps to about a 60 degree or 1 radian angle. The major sparse reward was given upon success in completing the task, this was given when the orientation of the hand was within the 0.5 dot

product threshold as calculated above, and when the end effector was within 15 cm of the objects centre. As the object has a radius and height of 8 cm, it is about 7 cm away.

There were 4 penalties calculated for each step. A large penalty was given when the object fell off the table. A smaller penalty was given when if the object fell over, which was calculated using the pitch orientation of the object. A penalty was given based on time; this is a commonly used penalty in the literature to increase the efficiency of the task (Zhu et al., 2019). Another major penalty used was to avoid collisions. To detect collisions, the Isaac sim contact sensor classes were used to detect collisions based on input meshes. Contact was detected for the pinky finger, the thumb and the wrist/forearm meshes. Penalties were given when the contact force for these meshes was above a threshold.

There were a few important hyperparameters that were tweaked throughout the development of the algorithm. My neural networks for the last report were both had 2 hidden layers of 32 neurons. For the current iteration this was increased to two hidden layers of 64 neurons. In a similar manipulation reinforcement learning study by Shahid et al. (2020), networks of two hidden layers containing 64 neurons were used. In the current experimentation, increasing the number of neurons increased the efficiency of the learning. The file size of the exported policy network was around 38 Kilobytes.

To train the agent, 64 parallel environments were run for 800 iterations. Compared to the last report, where only 600 iterations were performed, as the Markov Decision Process definition slightly changed with the addition of collision detection and the orientation of the hand, these extra iterations were needed to train the more complex environments and to properly identify whether the mean reward curve converged.

Another important hyperparameter in the algorithm was the RL horizon. In PPO algorithms, the horizon defines the number of future steps considered when calculating the return of action-state space. The length of the horizon is especially important during the exploration stage of learning, where the agent attempts register an optimal trajectory from its random steps. A horizon length of 32 steps was used. As the dense distance reward was significantly lower compared to the sparse rewards of the success conditions being met, it is important that the sparse reward was considered in the return calculations of past actions.

Simulation Environment

In the current solution, each environment contains a robot, a table and a beaker object. These were adapted Isaac lab documentation. The Fourier GR1 class and the table were from a Fourier GR1 Teleoperation example (NVIDIA, 2025). While the beaker was found using the Isaac Sim asset browser. The robot was configured as an instance of the Isaac Lab *Articulation* class, which allows joint movement toward target positions through a simple interface. This version of the robot has 7 DOF in the arms, and access to the finger joints which have 6 DOF in each hand.

The table and steering wheel were both configured as instances of the Isaac Lab *RigidObject* class. However, the table was configured as a kinematic body, as opposed to a dynamic body, meaning that it is not affected by physical forces such as gravity or collisions. It can only be

moved through direct scripting. The scale of the beaker was decreased so its dimensions were equal to the mugs used in Robocup@home, stored in the lab, they have a height and radius of 8 cm. The position of the beaker was randomised between a range of 40 cm by 30 cm on the table for each episode.

For each timestep in the simulation, the rewards were calculated, actions targets were calculated based on the observed state and termination conditions were checked. The output action targets from the policy were first normalised between -1 and 1 using the hyperbolic tan function, then outputs were scaled down based on the distance of the end effector to the object. In the original report, one of the major issues in the reinforcement learning algorithm was the behaviour of the agent immediately pushing the object off the table. In my original solution to combat this I added a penalty for the world frame velocity of the hand and added a condition in the termination that the velocity had to be below a threshold. However, after advice from Dr Francisco Cruz Naranjo, to simplify my reward function, I took the approach to scale down the speed based on the object to hand distance. This was more effective in reducing the number of attempts where the object was knocked off the table.

At each step episodes were terminated if the simulation went over the max time of 5 seconds, if the beaker fell off the table, and if the robot had completed its task. This was defined as the robot hand being within 15 cm of the object's centre while it's palm was facing the object (which was calculated as stated above).

To more simulate real world noise, where robot sensor readings, computer vision and joint motors are often unreliable, a noise of 5% was added to the observations before being input into the network, and to the actions before they were input into the robot joint target interface.

Results

There are many different methods to measure the success of the reinforcement learning, the most fundamental being observation. In Figure 1 I have linked a YouTube video of the final reinforcement learning training. In the video, the agents resetting visually indicates that they were successful in reaching out to the beaker. There are other behaviours that can be observed in the demonstration. From the starting position of the arm vertically down, the robot moves the arm outwards away from the table seemingly to avoid collisions. In earlier iterations of the algorithm, it was common for the robot to collide its thumb with the table when moving from the starting position by immediate raising its arm. The algorithm performed relatively quickly for 800 iterations of 64 parallel environments, taking around 30 minutes.

https://youtu.be/x_eXMO00iNc

Figure 1. Video of the final reinforcement learning training.

As we see from the video the reinforcement learning algorithm is behaving correctly and achieving its goal, we can look at the mean reward function to determine if the policy requires more iterations to reach the optimal solution. From figure 2, the mean reward curve starts plateauing after 400 iterations, where the variance decreases over time. This is indicative of the reinforcement learning converging with the optimal solution.

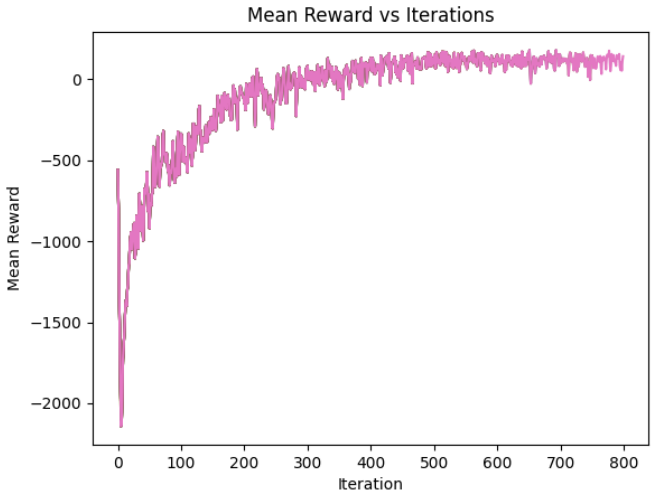

Figure 2. Mean reward of the algorithm vs the iterations.

I also used several other methods to determine if the algorithm was successful. By storing the number of successes, time outs, and episodes terminated when the object fell off the table, I was able to track the breakdown of episode terminations over the iterations. During the exploratory phase the dropped items and time outs are much higher, where around the 400 iterations mark the successes near 100%. This aligns well with the mean reward function.

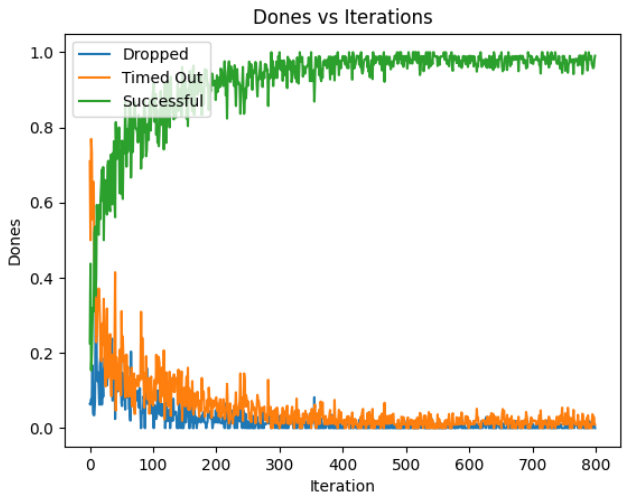

Figure 3. Breakdown of episode termination reason vs the iterations.

I found this success calculation to be somewhat biased, as successful episodes normally were under 1 second, while timeouts hit the 5 second time limit. This meant that when I was displaying my results to people the number of environments on screen at any one time that were failing was being underrepresented in the data. Therefore, I attempted to scale the termination breakdowns multiplying them by the length of the episode. In Table 1 I have listed the termination breakdowns for the final 2 iterations, comparing the time normalisation approach. The success rate is still high at 94.1% with an average time of around 0.99 seconds, while the attempts where the item was dropped averaged around 2.08 seconds.

Table 1. Breakdown of terminations in the final 2 iteration.

	Terminations (%)	Terminations scaled by the time of the episode (%)
Successes	98.5	94.1
Object Dropped	0.5	1.1
Time outs	1.0	4.8

To track the reward calculation over the course of the training I stored averaged values of each of the 7 reward components for the 64 environments at each step, then averaged them again for each iteration. Similar to the mean reward curve, many of the rewards plateaued over the course of the training.

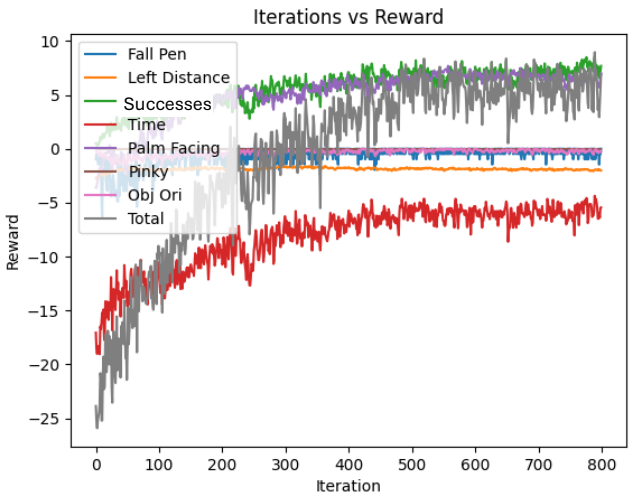

Figure 4. Breakdown of reward function over the course of the training.

Translating my solution to Isaac Sim and ROS

After successfully training the robot to reach towards an object with a randomised position, the next step was translating the policy into an Isaac Sim environment that could eventually be integrated with our ROS set up. However, I faced many issues when attempting to set up the simulation. The first major issue was setting up the docker container. Due to the python versions and ubuntu versions, it seems as though the most recent versions of Isaac sim are incompatible with ROS humble. In the current document for Isaac Sim version 5.1.0 there is a tutorial for ROS 2 development, however I couldn't create a docker container that could work through it due to the compatibility issues. Additionally on the NVIDIA docker platform, their most recent container for Isaac Sim and ROS development that was updated only 1 week ago, still uses the Isaac Sim version from June last year. As the reinforcement learning policy was for the most recent system, and as the previous versions of Isaac Sim are very under-documented, I changed my goal to just implement python scripts and classes in Isaac Sim that could replicate the environment and behave according to the policy.

I was able to download the Isaac Sim environment from the reinforcement learning as a USDA file (Human-readable USD). I was then able to develop Isaac Sim python scripts that could load the USD file stage and the robot into the simulation. In the USDA file created from the simulation, the Fourier GR1T2 USD was defined by a specific path. In my python Isaac Sim script, I was able to import this same USD file of the robot into the simulation. I then was able to write a python class that could load in the policy network, and had callbacks for the Isaac sim physics steps. I these callbacks, it calculates the observation state, passes it through the network and then inputs the action targets into an Isaac Sim interface to move the robot. However, when running the simulation, although the observation state calculations and policy were correct, the action targets did not align with the Isaac Lab results. In Figure 5 I have linked a video of the Isaac Sim environment. It seems that the robot is stuck in a loop. As the outputs of the policy are stochastic and continuous, an infinite loop is unexpected. One reasoning behind this abnormal behaviour could be with

Isaac Lab. The import for the Fourier GR1T2 in the Isaac Lab library used in the Teleoperation example (mentioned above) states it has 'High PD', this means that it has stronger stiffness and damping. These changed physics can cause the robot to enter a state it has never trained for and therefore exhibit unpredictable behaviours (i.e. a loop). There seems to be a method to import the USD files for the robot directly into Isaac Sim, however on attempting this method I faced issues with defining the actuators.

https://youtu.be/FXGksvHKOnU

Figure 5. Isaac Sim policy runner result.

Future Directions

There are many future directions to take the project in. First and foremost, either the simulation or reinforcement learning physics environment needs to be adjusted to be more cohesive. It may be beneficial to determine which will align more closely to the real-world robot. Another task is to create a ROS node. Currently I have set up docker files for ROS and Isaac sim with the slightly older Isaac Sim version, and it should be simple to map my current Isaac sim python scripts to work in the older version. Once that is completed, creating a ROS node to handle the policy is a good task. Using the actual joint target interface provided by Fourier may solve the issue of the mapping the Isaac Lab environment to Isaac Sim itself.

In the scope of reinforcement learning, there is a lot of space to build upon the reinforcement learning algorithm. One of the most important tasks that has been brought to my attention is the collision detection. The Fourier uses a depth cost map to avoid collisions, if a vision pipeline is integrated into the Isaac Lab environment, this could be used to inform the training.

References

Dohare, S., Lan, Q., & Mahmood, A. R. (2023). "Overcoming policy collapse in deep reinforcement learning. *Sixteenth European Workshop on Reinforcement Learning.*

Liu, R., Nageotte, F., Zanne, P., De Mathelin, M., & Dresp-Langley, B. (2021). Deep Reinforcement Learning for the Control of Robotic Manipulation: A Focussed Mini-Review. *Robotics*, *10*(1), 22. https://doi.org/10.3390/robotics10010022

NVIDIA. (2025, October 23). *Available Environments — Isaac Lab Documentation*. Isaac Lab Documentation. Retrieved October 23, 2025, from https://isaac-sim.github.io/IsaacLab/main/source/overview/environments.html

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy optimization Algorithms. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.1707.06347

Shahid, A. A., Roveda, L., Piga, D., & Braghin, F. (2020). Learning Continuous Control Actions for Robotic Grasping with Reinforcement Learning. *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 4066–4072. https://doi.org/10.1109/smc42975.2020.9282951

Tan, C. (2025). Comparative study of reinforcement learning performance based on PPO and DQN algorithms. *Applied and Computational Engineering*, *175*(1), 30–36. https://doi.org/10.54254/2755-2721/2025.ast24879

Tang, W., Cheng, C., Ai, H., & Chen, L. (2022). Dual-Arm Robot Trajectory Planning Based on Deep Reinforcement Learning under Complex Environment. *Micromachines*, *13*(4), 564. https://doi.org/10.3390/mi13040564

Appendix

My work is available in the rUNSWeep-fourier git repository, in branch ‘manip-rl-isaac’, in the folder ‘manip-rl’. This contains two folders, one for the Isaac Lab reinforcement learning environment, one for the Isaac Sim python scripts. README.md files are available in both folders detailing how to run each. I have written documentation for my scripts, but also for my docker containers.